INVESTOR, TECHNICAL & UNIT-ECONOMICS DILIGENCE BRIEFING | TRACK 1: MERCHANT GROWTH AI

PaytmAI VyaparVoice: Investor, Technical & Economics Deep-Dive Handbook

- **Product:** PaytmAI VyaparVoice (`v1.5.0` / `v1.0-finals` , Track 1: Merchant Growth AI)
- **Team & Founder:** SemiTechnical | Kevin Joseph
- **Live Cloud Prototype:** https://paytmai-vyaparvoice.onrender.com/
- **Source Repository:** https://github.com/somebeing2/paytmai-vyaparvoice
- **Purpose of This Document:** Comprehensive founder-level technical, analytical, and unit-economics briefing prepared for investor, product, and technical due diligence. Every statement explicitly distinguishes what is **implemented in code**, what is **synthetic or simulated**, what is a **modelled financial assumption**, and what is **proposed for production validation**.

1. Executive Thesis: What Problem We Solve and Why Existing Tools Fail

1.1 The Silent Economics of Neighbourhood Merchant Churn

For high-frequency neighbourhood food, beverage, and provision counters (darshinis, tiffin rooms, bakeries, sweet shops, and kiranas), revenue is heavily concentrated in repeat local buyers. In our deterministic 60-day synthetic store ledger (*Sri Krishna Darshini & Provisions*, Indiranagar, `seed = 20261025`), `175` of `240` unique QR payers (`72.9%`) scan three or more times, generating `Rs 2,89,175` of the store's `Rs 3,06,365` 60-day QR GMV (`94.4%` of total sales; `Rs 1,74,685` across `917` scans in the most recent 30 days).

Despite processing hundreds of digital payments per week, small merchants lack retention visibility and safe promotional execution for three structural reasons:

1. **Transaction Streams Hide Payer Drift:** Standard UPI QR notifications confirm individual rupee amounts in real time (`assumption, to verify with Paytm`), but never alert the shopkeeper when `48` evening regulars silently stop scanning over `16` to `38` days, or when `2` every-two-day regulars vanish right after a congested morning queue.
2. **Uncapped Discount Liability:** Neighbourhood merchants operate on tight working capital and `15%` to `35%` gross margins. Traditional percentage or flat coupon blasts carry open-ended redemption risk: if `100` buyers redeem a `Rs 25` discount in one evening, `Rs 2,500` of margin disappears immediately.
3. **Subsidizing Organic Walk-Ins:** Without a randomized non-offer holdout group, merchants and platforms count every redeemed coupon as "growth", even when regular buyers would have visited and paid full price anyway.

1.2 The VyaparVoice Product Thesis

PaytmAI VyaparVoice converts passive longitudinal UPI QR scan logs into a closed-loop customer retention copilot governed by four steps: **Detect, Propose, Cap, and Measure**.

- **Core Value:** Automated churn and dead-hour detection, hard rupee stop-loss enforcement (`BEGIN IMMEDIATE` database locks), and unbiased `80/20` holdout profit accounting.
- **Voice as the Delivery Layer (Not the Problem Statement):** Rather than asking a counter-bound shopkeeper to study charts at `8:30 PM` closing time, VyaparVoice delivers a 3-sentence closing briefing and live `50%` , `80%` , and `100%` budget alerts out loud in **Kannada, Hindi, or English**.
- **Hardware Boundary:** To our knowledge the standard Paytm Soundbox is speaker-based with no microphone; this is to be validated with Paytm. Accordingly, Soundbox is treated strictly as the audio output speaker (`[PROPOSED hardware push; BUILT in web audio]`), while spoken merchant replies and budget commands are proposed through the **Paytm for Business phone app** (or a future voice-enabled device), with the browser microphone standing in during the prototype demo.

2. Complete Technical Stack & Codebase Architecture

2.1 Deliberate Zero-Bloat Stack (`~55 MB` RAM Footprint)

A key engineering decision for investor diligence is dependency minimalism. `requirements.txt` contains **only five runtime packages**:

- `fastapi>=0.110.0` (HTTP API, session middleware, static asset serving, OpenAPI)
- `uvicorn>=0.29.0` (ASGI server)
- `pydantic>=2.6.0` (request/response schema validation)
- `pyyaml>=6.0.1` (typed configuration loading from `config/default.yaml` and `config/sarvam.yaml`)
- `httpx>=0.27.0` (bounded-timeout REST client for Sarvam AI and Groq APIs)

All statistical tests, confidence intervals, Bayesian Monte Carlo draws, SQLite concurrency locks, sliding-window IP rate limiters, constant-time `hmac.compare_digest` PIN checks, and vanilla multilingual UI rendering are written in **standard-library Python (`3.11+`) and zero-build ES6 JavaScript**. Avoiding `150+ MB` of `numpy` / `scipy` / `pandas` wheels keeps worker RSS near `55.4 MB`, allowing `64` isolated concurrent demo sessions inside a `512 MB` container.

2.2 Module-by-Module Codebase Map

File / Module	Lines	Core Responsibility in `v1.5.0`
`app/main.py`	~760	FastAPI routes (`/api/overview`, `/api/briefing`, `/api/campaigns/*`, `/api/voice/*`, `/api/why/*`, `/health`), session middleware, IP rate limiter, `DEMO_PIN` unlock/lock endpoints.
`app/database.py`	418	SQLite connection factory (`PRAGMA synchronous=NORMAL`), per-DB `RLock` registry, `ContextVar` session routing, deterministic 60-day store seeder (`seed = 20261025`).
`app/engine.py`	2,831	KPI SQL aggregations, RFM segment queries, delayed-regular rush detector, verbal budget parser (`en` / `hi` / `kn`), atomic pass redemption (`BEGIN IMMEDIATE`), `50/80/100%` alerts, `80/20` ITT simulation, 6-rule guardrail engine, zero-PII fact builder.
`app/stats.py`	852	Pure-Python Acklam inverse normal quantile (`<1.15e-9` error), Wilson 95% score CIs, exact 2x2 hypergeometric Fisher test, binomial cap-overrun tail risk, `Beta(1,1)` Bayesian posterior (`10,000` draws), break-even solver, `6x4` sensitivity grid, `50`-shop power table.
`app/explain_simply.py`	323	Jargon-free plain-language outcome generator (`en` / `hi` / `kn`), Python integer fact extractor, banned-jargon linter, LLM numeric validator (`validate_llm_plain_output`).
`app/providers/router.py`	~520	Unified 3-tier `ProviderRouter` (`Sarvam -> Groq -> Browser Web Speech + Templates`), silent failover, call counters (`60` global / `35` session), fallback audit notices.
`app/providers/sarvam.py` & `groq_qa.py`	~1,300	`bulbul:v3` TTS, `saaras:v4` STT, `sarvam-105b-conversations` Chat, Groq `gpt-oss-20b` / `whisper-large-v3-turbo` / `orpheus-v1-english`, deterministic Q&A intent matcher, 3-script `verify_numeric_grounding`.
`app/providers/persistent_cache.py`	~210	SHA-256 SQLite `PersistentProviderCache` (`data/provider_cache.db`) keyed on `(provider, language, voice, exact_text)` + startup seeding from `assets/prewarmed/`.
`app/sessions.py` & `app/security.py`	~310	Per-visitor `233 KB` SQLite cloning (`30m` TTL, `64`-session LRU, `32 MB` total disk cap), sliding-window IP limiter (`30 req/min`), signed session unlock state.
`app/static/index.html` & `i18n.js`	~2,400	Zero-build trilingual (`kn`, `hi`, `en`) `Merchant View` and English `Technical & Audit View`, Web Speech wrapper, `10s` MediaRecorder clip cap, `Why?` provenance drawers.
`scripts/preflight.py` & `tests/*.py`	~2,600	`12` automated preflight gates, secret scanner, unit economics generator (`scripts/unit_economics.py`), `7` test suites (`311` `assert` statements) + `21/21` QA battery.

3. Deep Dive: Does VyaparVoice Use RFM Analysis, How Does Segmentation Work, and What Value Can Full RFM Add?

Investors and data scientists frequently ask whether our customer engine uses **RFM (Recency, Frequency, Monetary) Analysis**. Here is the exact source-code reality so you can answer with complete precision.

3.1 What Is Implemented Today: Exact R, F, M Feature Extraction + Behavioural & Cadence Rules

Short Answer: Yes, the SQLite engine explicitly computes **Recency (`R`), Frequency (`F`), and Monetary (`M`)** metrics for every payer and cohort, **augmented with Time-of-Day Daypart and Inter-Visit Gap Cadence**. However, it intentionally uses **domain-specific threshold and cadence rules rather than naive `1-to-5` e-commerce quintile binning** (`R=1..5, F=1..5, M=1..5`).

In `app/engine.py:get_dashboard_overview()` (`lines 640-660`), the database executes this exact per-customer R/F/M aggregation over non-simulated QR scans:

- **Recency (`R`):** `MIN(days_ago) AS min_days_ago` (days elapsed since the payer's most recent QR scan).
- **Frequency (`F`):** `COUNT(*) AS visit_count` (number of QR scans over the 60-day window).
- **Monetary (`M`):** `SUM(amount) AS total_spend` (cumulative rupee spend, plus `total_spend / visit_count` average ticket).

Across the 60-day synthetic ledger (`240` customers, `1,613` scans, `Rs 3,06,365` total GMV), the four cohorts in `customers` + `transactions` exhibit the following exact R, F, and M profiles:

Segment Key & UI Label	Buyers (`Eligible`)	Avg Recency (`R`)	Avg Frequency (`F`)	60d Monetary (`M`)	Avg Spend / Buyer	Action Mapped in Engine
`loyal_champion` (Frequent Regulars)	`72` (`55`)	**`2` days**	**`12.4` visits**	**`Rs 1,69,240`** (`55.2%` GMV)	`Rs 2,351`	Protect queue speed; **do not** blanket-discount
`at_risk_regular` (Silent Regulars)	`48` (`40`)	**`21` days** (`16-38d`)	**`7.7` visits**	**`Rs 72,755`** (`23.7%` GMV)	`Rs 1,516`	`CMP-WINBACK-01` : `Rs 25` off `Rs 150+` (`Rs 300` cap)
`afternoon_snack` (3-5 PM Visitors)	`55` (`34`)	**`11` days**	**`4.7` visits**	**`Rs 47,180`** (`15.4%` GMV)	`Rs 858`	`CMP-DEADHOUR-02` : `Rs 20` off `Rs 110+` (`Rs 240` cap)
`new_or_casual` (First / Casual)	`65` (`54`)	**`7` days**	**`1.4` visits**	**`Rs 17,190`** (`5.6%` GMV)	`Rs 264`	Second-visit nudge opportunity (tracked, unactivated)
Total Synthetic Store	**`240` (`183`)**	**Weighted**	**`6.7` visits**	**`Rs 3,06,365`** (`100%`)	**`Rs 1,277`**	Policy filter: `dpdp_consent = 1` & `cooldown >= 14d`

3.2 Beyond Static RFM: Cadence-Normalized & Queue-Context Detection (`OPS-CARE-02`)

Why is classic `1-to-5` Quintile RFM insufficient on its own for neighbourhood food and grocery retail?

In standard e-commerce quintile RFM, `Recency = 8 days` scores in the **top healthy quintile (`R = 5`)** because casual shoppers visit every two weeks. Yet at a Bengaluru darshini, a morning filter-coffee regular who scans **every `2` days** is already **`4x` overdue at Day `8`** and actively switching to a competing corner shop.

To catch this early, `detect_delayed_regulars_from_ledger()` (`app/engine.py:2029`) implements a **pure transaction-log cadence + rush-density detector** with four conjunctive rules:

1. **High Frequency (`F >= 12`):** Filters for habitual store anchors (`16` visits each for `CUST-009` Sneha and `CUST-038` Divya).
2. **Tight Inter-Visit Cadence (`median_gap in [1.5, 3.0]` days and `mean_gap <= 3.0` days):** Computes the sorted inter-arrival day gaps `[days_sorted[i+1] - days_sorted[i]]` per buyer (`2.0` -day median, `2.5` -day mean = `12` visits/month).
3. **Relative Recency Anomaly (`R >= 8` days):** Flags buyers whose current silence (`8` days) exceeds `~3x to 4x` their personal median cadence.
4. **Queue Congestion Context (`slot_scans >= 6` in a peak hour):** Joins the customer's final scan timestamp against hourly store scan density, flagging cases where their last visit coincided with a congested peak rush (`8` scans in the `7-8 AM` morning slot),

framed transparently as an inferred cause (`"possibly after a busy rush"`).

5. **Monetary Risk Valuation (`M`):** Multiplies flagged count (`2`) x rounded historical average bill (`Rs 205` , from `Rs 6,550 / 32 = Rs 204.69`) x monthly visit cadence (`30 / 2.5 = 12`) = **`Rs 4,920` Monthly spend at risk** (explicitly labelled *historical spend, not guaranteed lift*).

3.3 High-Value Production Upgrades Using Continuous RFM (`[INVESTOR ROADMAP VALUE-ADD]`)

In the current prototype, the four broad segment labels (`archetype`) are assigned at seed time while `detect_delayed_regulars_from_ledger` runs dynamically on raw transactions. Upgrading the production engine to a **Dynamic Cadence-Normalized RFM + Daypart Matrix** unlocks four concrete commercial gains without adding ML black boxes:

1. **Dynamic Nightly State Transitions (Zero Static Labels):** A nightly SQL materialization computes `(R_days, median_gap, R_ratio = R_days / median_gap, F_60d, M_avg_bill, M_total_60d, modal_hour_slot)` on live Paytm QR streams. Buyers automatically move from `New (F=1)` -> `Developing (F=2..4)` -> `Habitual Champion (F>=8, R_ratio < 2.0)` -> `Early Drift (R_ratio 2.5..4.5)` -> `Lapsed Win-Back (R_days 16..45)` -> `Dormant (>60d, suppress spend)` .
2. **Monetary (`M`) Personalized Minimum-Bill Stretching (Margin Protection + Basket Lift):** Currently, all `32` buyers in `CMP-WINBACK-01` receive the same `Rs 25` off `Rs 150+` offer. Using individual Monetary ticket `M_i` lets VyaparVoice set each buyer's minimum bill to `ROUND_TO_10(1.25 * M_i)` while capping reward at `<= 15%` of the stretched bill:
   - A `Rs 120` snack regular gets **`Rs 20` off `Rs 150+`** (lifting basket by `+Rs 30`).
   - A `Rs 280` family parcel regular gets **`Rs 35` off `Rs 350+`** instead of getting a free `Rs 25` subsidy on a `Rs 280` bill they already spend.
   - A `15%` -margin kirana regular with `M_i = Rs 220` gets **`Rs 12` off `Rs 275+`** (`4.4%` effective discount), turning negative kirana promo ROI positive.
3. **Cap-Constrained Priority Dispatch (`RFM Value-at-Risk Ranking`):** When `40` buyers are eligible but the merchant's `Rs 300` cap only funds `12` redemptions, ranking treatment selection or wave priority by `Expected Recoverable Margin = F_monthly * M_avg * shop_margin * P_decay(R_ratio)` ensures scarce promo rupees target high-value regulars before 45-day cold accounts.
4. **Churn-Adjusted Merchant Credit Underwriting Signal (`[OPEN ASSUMPTION WITH PAYTM]`):** Aggregate store-level RFM stability (for example, what share of a shop's GMV comes from `F >= 8, R_ratio < 2.0` habitual regulars versus one-time footfall) is a much stronger leading indicator of 90-day cashflow resilience than raw trailing 30-day GMV alone.

4. Causal Holdout Measurement, Statistical Engine & Atomic Spend Locks

4.1 Intention-to-Treat (`ITT`) `80/20` Holdout Accounting

Every campaign splits policy-eligible buyers (`dpdp_consent = 1` and `last_promo_days_ago >= 14`) deterministically `80/20` (`default_allocation_ratio = 4.0`):

- For `CMP-WINBACK-01` : `48` silent regulars -> `43` DPDP opted-in -> `40` past `14` -day cooldown -> **`32` Treatment** (`Rs 25` pass on `Rs 150+`) vs **`8` Holdout** (`Rs 0` offer, zero message).
- **Why `ITT` Matters to Technical Judges & Investors:** Even if the `Rs 300` cap pauses cashback after `12` bills and a 13th treatment buyer visits at regular price (`REDEEMED_CAP_REFUSED`), `ITT` evaluates **all `32` assigned treatment buyers against all `8` holdout buyers** so early cap exhaustion never biases estimated return rates.

4.2 Exact Source-Code Logic of the 48-Hour Simulation (`Illustrative, Not Evidence of Lift`)

In `app/engine.py` (`_draw_independent_outcomes` , `lines 1613-1652`), two simulation modes exist:

- **Canonical Demo Mode (`mode="canonical"` , default on UI button):** Uses fixed deterministic counts so every judge and audit screen sees identical arithmetic: `12` of `32` treatment buyers return (`37.5%`) with deterministic bills `210 + ((idx % 4) * 15) - 15` (`Rs 195, 210, 225, 240` x `3` = `Rs 2,610` , mean `Rs 217.50`), vs `1` of `8` holdout buyers (`12.5%`).
- **Stochastic Sensitivity Mode (`mode="stochastic"`):** Seeds `random.Random(f"{campaign_id}:{seed}:{offer_rupees}")` (`seed = 20261025`) with assumed baseline `p_base = 0.125` and assumed response curve `p_treat = p_base + (0.50 - p_base) * (1 - exp(-offer_rupees / 22.0))` (`38.0%` at `Rs 25`).

4.3 Step-by-Step Profit Waterfall & Statistical Guardrails (`CMP-WINBACK-01`)

1. **Expected Organic Returners in Treatment:** `(1 / 8) * 32 = 4.0` visits (`4 * Rs 217.50 = Rs 870.00` organic sales; absorbs `4 * Rs 25 = Rs 100` organic cashback subsidy).
2. **Net Incremental Visits & Sales (`Grade B`):** `12 - 4 = 8` extra visits x `Rs 217.50` = **`Rs 1,740.00` extra sales** (`5.8x` gross sales multiple on `Rs 300` cashback).
3. **Merchant Net Profit Before Fee (`Grade C` , assumed `30%` margin):** `(Rs 1,740 * 0.30) - Rs 300 = Rs 522 - Rs 300` = **`+Rs 222.00`** (`+74.0%` ROI on cashback).
4. **Merchant Net Profit After Assumed `5%` Fee (`Grade C`):** Levying an assumed `5.0%` promo fee (`assumption, to verify with Paytm`) across **all `Rs 2,610.00` redeemed campaign bills** (`Rs 130.50` , conservatively charging fee even on the `4` organic redeemers) leaves **`+Rs 91.50`** (`+30.5%` net ROI; rounded to `Rs 92` in `Explain Simply`).
5. **Statistical Honesty (`app/stats.py`):**
   - **Wilson 95% Score Intervals:** Treatment `37.5%` (`[22.93%, 54.75%]`) vs Holdout `12.5%` (`[2.24%, 47.09%]`).
   - **Exact 2x2 Hypergeometric Fisher Test:** Two-sided `p = 0.2364` (one-sided `p = 0.1895`). Single-store `n = 40` is explicitly marked **underpowered and directional only** (`pilot_advancement_rule_met = False`).
   - **Conjugate `Beta(1,1)` Bayesian Posterior (`10,000` draws, `seed = 20261025`):** `P(Lift > 0) = 88.8%` , `P(Pre-Fee Profit > 0) = 67.0%` , `P(Post-Fee Profit > 0) = 52.1%` , 95% credible interval `[-Rs 19.97, +Rs 14.73]` per nudged buyer.
   - **Pooled Multi-Shop Pilot Power (`compute_pooled_pilot_power_table`):** Pooling `50` opt-in shops (`n = 2,000` : `1,600` treatment, `400` holdout) against `12.5%` baseline achieves **`83.3%` power at `+6 pp` true lift** and **`92.5%` power at `+7 pp` true lift** (`alpha = 0.05`).

4.4 Atomic Spend Cap & Binomial Overrun Protection

- **Pre-Launch Binomial Overrun Warning:** Issuing `32` passes of `Rs 25` creates `Rs 800` face exposure against a `Rs 300` (`12` -bill) cap. At `p = 0.375` , `P(Binomial(32, 0.375) > 12) = 42.2%` (`> 10%` warning threshold), so the UI proactively recommends raising the cap to **`Rs 400`** (`16` bills, `5.2%` overrun probability).
- **Runtime Atomic Stop-Loss (`redeem_single_pass_atomic`):** Executes `conn.execute("BEGIN IMMEDIATE")` inside SQLite before checking `budget_spent + offer_rupees <= max_budget` . Parallel QR scans serialize cleanly: bill `#12` hits `Rs 300` , fires the one-time `100%` spoken alert (`campaign_threshold_alerts`), transitions status to `CAP_PAUSED` , and bills buyer `#13` at regular price with `Rs 0` cashback.

5. Voice, Multilingual AI Stack & Zero-Hallucination Architecture

5.1 Three-Tier Failover Chain (`app/providers/router.py`)

1. **Tier 1 Primary - Sarvam AI (`api-subscription-key`):**
   - `bulbul:v3` (`POST /text-to-speech` , `22050 Hz` WAV, speaker `priya` for `en-IN` / `kn-IN` , `shubh` for `hi-IN`).
   - `saaras:v4` (`POST /speech-to-text` , browser clips hard-capped at `10` seconds).
   - `sarvam-105b-conversations` (`POST /v1/chat/completions` , `temperature=0.1` , `max_tokens=110`).
2. **Tier 2 Secondary - Groq (`Authorization: Bearer`):**
   - `openai/gpt-oss-20b` (chat failover), `whisper-large-v3-turbo` (multilingual STT failover), `canopylabs/orpheus-v1-english` (English TTS failover; `hi` / `kn` TTS step directly to Tier 3 if Sarvam TTS is down).
3. **Tier 3 Zero-Cost Fallback - Browser Web Speech + Deterministic SQLite Templates:**
   - `window.speechSynthesis` and `SpeechRecognition` paired with parameterised `en` / `hi` / `kn` templates in `app/templates.py` , `app/providers/groq_qa.py` , and `app/explain_simply.py` . Works offline or with zero API keys.

5.2 Five Hard Gates Preventing Hallucination or PII Leakage

1. **Deterministic Intent Fast-Path (`0` LLM Calls):** `detect_common_qa_intent` intercepts `customers today` , `profit` , `repeat` / `total customers` , `spend & remaining` , `sales summary` , `why 40` , `max loss` , `after cap` , `milk` , and `help` in `en` / `hi` / `kn` and answers straight from SQLite.
2. **Zero-PII Sanitizer:** `build_sanitized_store_facts` + `assert_zero_pii` strip all customer names, IDs, masked phones, and UPI handles before any external HTTP payload is constructed.

3. **Three-Script Numeric Grounding Gate:** `verify_numeric_grounding` and `validate_llm_plain_output` translate Devanagari (`०-९`) and Kannada (`೦-೯`) digits to ASCII (`0-9`) and reject any LLM sentence containing even one number absent from the SQLite fact set, falling back immediately to a deterministic template.
4. **Two-Step Verbal Budget Confirmation:** `parse_verbal_budget_command` parses `en` / `hi` / `kn` rupee words/digits (`Rs 50` to `Rs 2,500`) and time windows (`today` , `48 hours` , `this weekend` , `this week` , `this month`), writing a pending row to `pending_confirmations` and committing to `campaigns` + `audit_logs` only after explicit `"Yes"` confirmation.
5. **6-Rule Policy & Prompt-Injection Blocker (`evaluate_guardrail_attack`):** Blocks prompt-injection strings, non-consented DPDP targets, `< 14` -day cooldown spam, `> 22%` effective discounts, out-of-range caps, and 0%-holdout blasts before any model or DB write.

6. Deployment Cost Breakdown & Main Cost Drivers

Investors want to know **what this costs to run today, what drives variable cost per merchant, and what infrastructure costs look like from a 50-shop pilot to 10,000+ merchants**.

6.1 Stage 1: Current Hackathon Cloud Deployment (`$0 / Month` Fixed Infra)

- **Compute & Hosting (`render.yaml`):** Hosted on Render Web Service `plan: free` (`0.1` shared vCPU, `512 MB` RAM, auto-TLS, sleeps after `15` minutes idle; wakes in ~20-35 seconds on `/health`). Monthly hosting bill: **`Rs 0 ($0)`** .
- **AI & Voice Spend on Public URL (`PUBLIC_DEPLOY=true`):** Anonymous public visitors consume **`Rs 0`** in live API calls because live provider calls stay locked behind `DEMO_PIN` (`hmac.compare_digest`), `60` global / `35` session Sarvam call caps, `30%` (`150` -credit) ledger ceiling on a `500` -credit account, `30 req/min` IP rate limits, and `PersistentProviderCache` .

6.2 Per-Merchant Monthly Variable Cost Stack (`scripts/unit_economics.py` - **`MODELLED ASSUMPTIONS`**)

Configured in `config/default.yaml` and `config/sarvam.yaml` , our reproducible unit-economics script models steady-state per-merchant variable COGS (`~0.78` retention waves/month under the `14` -day cooldown):

Cost Component	Uncached / Mo	Cached / Mo	Share of Cached COGS	Primary Cost Driver & Architectural Lever
1. Customer Offer Delivery (WhatsApp / Push)	`Rs 43.00`	**`Rs 43.00`**	**`89.6%`**	**#1 Cost Driver (`MODELLED ASSUMPTION`):** Models paid WhatsApp utility/offer templates (the demo app does **not** send WhatsApp/push messages). Replacing third-party WhatsApp with native **Paytm consumer-app push notifications** (`assumption, to verify with Paytm`) could compress this cost by `80%+` toward `~Rs 2-5/mo` .
2. Voice TTS Synthesis (`Sarvam bulbul:v3`)	`Rs 27.00`	**`Rs 3.20`**	**`6.7%`**	**#2 Cost Driver (Solved by Caching):** Configured at `Rs 30 / 10k chars` (`config/sarvam.yaml`). Without caching, daily closing briefings + alerts cost `~Rs 27/mo` . `PersistentProviderCache` achieves an `~88.1%` hit rate on recurring phrases/numbers, cutting TTS cost by **`88%`** (**`-Rs 23.80/mo`**).
3. Voice STT (`saaras:v4`) + SLM / Intent Compute	`Rs 1.80`	**`Rs 1.80`**	**`3.7%`**	Configured STT at `Rs 30 / hour` (`5s` average clip = `~Rs 0.04/command`) + deterministic SQLite intent routing (`80%+` of merchant questions hit `0` LLM tokens).
Total Modelled Variable COGS / Merchant	**`Rs 71.80`**	**`Rs 48.00`**	**`100.0%`**	**`Rs 5.00 / merchant / month`** if delivered via native Paytm app push instead of paid WhatsApp (`modelled sensitivity`).

6.3 Infrastructure Cost by Deployment Stage (`INDICATIVE ENGINEERING ESTIMATES - ASSUMPTION`)

Because the core engine is deterministic SQL + lightweight REST routing (no dedicated GPU servers to rent), fixed cloud infrastructure scales very cheaply:

1. **50-Shop Controlled Validation Pilot (`2,000` tracked buyers, ~3 months):**
   - **Cloud Compute & DB:** 1 always-on container (`Render Starter $7/mo` or `Standard $25/mo` , `0.5-1` vCPU, `512 MB - 2 GB` RAM so there is zero `15m` cold sleep) + 1 small managed PostgreSQL instance (`$7-$15/mo`) + persistent disk/S3 audio cache (`~$1-$2/mo`) = **`~$15 to $42 / month`** (**`~Rs 1,300 to Rs 3,600 / month`** total fixed infra).

- **Variable Voice + Messaging (`50` shops):** `50 x Rs 5.00` (voice/LLM only, `~Rs 250/mo`) up to `50 x Rs 48.00` (if paid WhatsApp is used, `~Rs 2,400/mo`).
- **Total 50-Shop Pilot Tech Budget:** Roughly **`Rs 4,000 to Rs 6,500 / month`** all-in (excluding merchant-funded or pilot-subsidized `Rs 300` cashback budgets; `50` shops x `Rs 300` cap/wave = `Rs 15,000` maximum cashback pool per wave if pilot-subsidized).

2. **10,000-Merchant Early Production Rollout (`INDICATIVE ESTIMATE - ASSUMPTION`):**
   - **Cloud Compute, DB & Cache:** 2-4 horizontally scaled stateless API workers + partitioned managed PostgreSQL + Redis atomic cap counters + CDN/S3 shared audio cache = **`~Rs 25,000 to Rs 55,000 / month` fixed infra** (`~Rs 2.50 to Rs 5.50 / merchant / month`).
   - **Dominant Scale Cost Insight for Investors:** At `10,000+` merchants, **server infrastructure is `< 10%` of total COGS**. The business model lives or dies on two unit levers: (a) keeping voice phrase cache hit rates high (`>85%` , already built via `PersistentProviderCache`) and (b) delivering customer offer passes via **zero-marginal-cost Paytm app push notifications** rather than third-party WhatsApp fees (`assumption, to verify with Paytm`).

7. Store Economics, Margin Limits & Modelled Monetization

7.1 Why Beachhead GTM Must Target `~30%` Margin Food Counters Before Kiranas

From `scripts/unit_economics.py` (`CMP-WINBACK-01` simulated wave: `Rs 1,740` incremental sales, `Rs 300` cashback cap, `Rs 2,610` total redeemed bills):

- **Minimum Store Gross Margin to Break Even: `17.2%` pre-fee** (`Rs 300 / Rs 1,740`) and **`24.7%` post-5%-fee** (`(Rs 300 + Rs 130.50) / Rs 1,740`).
- **Store Net Profit Across Margin x Assumed Promo Fee (`Grade C` Sensitivity):**
  - **15% Margin (Packaged Kirana Staples):** `-Rs 39.00` at `0%` fee | **`-Rs 169.50`** at `5%` fee.
  - **20% Margin (General Grocery):** `+Rs 48.00` at `0%` fee | **`-Rs 82.50`** at `5%` fee.
  - **25% Margin (Bakery / Dairy / Snacks):** `+Rs 135.00` at `0%` fee | **`+Rs 4.50`** at `5%` fee.
  - **30% Margin (Darshini / QSR Base Case):** `+Rs 222.00` at `0%` fee | **`+Rs 91.50`** at `5%` fee.
  - **35% Margin (Sweet Shop / Beverages):** `+Rs 309.00` at `0%` fee | **`+Rs 178.50`** at `5%` fee.

Investor Takeaway: A `Rs 25` off `Rs 150` (`16.7%`) pass is profitable post-fee for **`28%` to `40%` gross margin food/beverage merchants** (our Phase 1 beachhead), whereas `15%` to `20%` kiranas require category-preset passes (`Rs 10` to `Rs 15` on `Rs 250+` bills = `4%` to `6%` effective discount, paired with `0%` SaaS-only pricing rather than a `5%` bill-volume fee).

7.2 Modelled Platform Revenue Scenarios (`OPEN ASSUMPTIONS TO VERIFY WITH PAYTM`)

`scripts/unit_economics.py` evaluates three illustrative per-merchant monthly revenue structures against `Rs 48.00/mo` cached COGS (`Rs 71.80` uncached):

- **Case C - Voice Copilot SaaS Add-on Only (`Rs 79/mo` assumed):** `Rs 31.00/mo` gross profit (`39.2%` cached gross margin; `9.1%` uncached). Fits low-margin kiranas where bill-volume promo fees hurt merchant ROI.
- **Case B - SaaS (`Rs 79`) + Assumed `5%` Promo Fee (`Rs 102/mo` at `~0.78` steady waves/mo = `Rs 181/mo`):** `Rs 133.00/mo` gross profit (**`73.5%` cached gross margin**; `60.3%` uncached).
- **Case A - Full Stack Including Working-Capital Referral Assumption (`Rs 181 + Rs 77 = Rs 258/mo`):** `Rs 210.00/mo` gross profit (**`81.4%` cached gross margin**). Treated strictly as a long-term ecosystem sensitivity, not near-term product revenue.

8. Value Addition, Competitive Defensibility & Phased Roadmap

8.1 Five Layers of Value Addition

1. **Turns Passive Payment Rails Into Retention Intelligence:** Extracts R, F, M, inter-visit cadence, and rush-hour drop-offs from existing QR scan timestamps with zero POS integration or cashier data entry.
2. **Eliminates Merchant Fear via Hard Stop-Loss Engineering:** Atomic cap enforcement (`Rs 300` hard ceiling on `Rs 800` issued pass value) makes retention spend as predictable as buying milk stock.
3. **Self-Auditing ROI via Mandatory `80/20` Holdouts:** Prevents false coupon attribution by continuously subtracting organic baseline returners before reporting merchant profit.

4. **Fits Single-Counter Closing Habits:** Spoken 3-sentence Kannada/Hindi/English briefings, two-step verbal budget confirmation, and jargon-free `Explain Simply` readouts remove chart friction.
5. **Zero-Hallucination Financial AI Pattern:** Separates deterministic Python math from neural speech/phrasing via a 3-script digit grounding gate and immutable audit log.

8.2 Three-Phase Validation & Production Roadmap

- **Phase 1 (Weeks 1-6) - Controlled `50` -Shop Darshini Pilot:** Native Kannada/Hindi speaker review; validate Soundbox audio push + Paytm for Business app push-to-talk/tap input with Paytm; run `4` weekly waves (`n = 2,000` buyers, `80/20` holdout) with hard stop rules (halt if pooled post-fee profit 95% lower bound `<= Rs 0`, cap overruns `> 0`, or opt-outs spike).
- **Phase 2 (Months 2-4) - Dynamic Continuous RFM & Category Margin Presets:** Materialize nightly `R_ratio = R / median_gap` state transitions, individual Monetary (`M`) basket-stretch hurdles (`1.25x` avg bill), and category margin presets (`darshini 30%` vs `kirana 15-18%`).
- **Phase 3 (Months 4-6) - Partitioned Production Scale & Native Push Integration:** Migrate from isolated SQLite files to partitioned PostgreSQL + Redis atomic cap counters and replace modelled `Rs 43/mo` external messaging with native in-app push notifications (`assumption, to verify with Paytm`).

9. Honest Diligence Risk Register (What Is Not Claimed Today)

Risk / Boundary	Current Prototype Reality (`v1.5.0`)	Mitigation / Validation Path
Synthetic Data & Simulated Lift	All `240` buyers, `1,613` scans, and `12/32` vs `1/8` 48h returns come from `seed = 20261025`, not real stores.	Run Phase 1 `50` -shop (`n = 2,000`) randomized pilot before claiming empirical lift.
Soundbox Microphone Assumption	To our knowledge the standard Paytm Soundbox is speaker-based with no microphone; this is to be validated with Paytm.	Route spoken replies through Paytm for Business phone app (browser mic in demo).
Single-Wave Statistical Noise	One `40` -buyer wave yields Fisher `p = 0.2364` and Bayesian `P(post-fee > 0) = 52.1%`.	Enforce multi-wave / `50` -shop pooling (`92.5%` power at `+7 pp`) and `Grade A/B/C` badges.
Kirana Margin Compression	`Rs 25` off `Rs 150` loses `-Rs 82.50` at `20%` margin under an assumed `5%` fee.	Restrict initial pilot to `~30%` margin food shops; ship `4-6%` stretched-basket presets + SaaS-only pricing for kiranas.
Draft Localisation	Hindi and Kannada strings are AI-assisted drafts (`Draft - unreviewed by native speaker`).	Complete native merchant copy review prior to any live shop test.
Demo Hosting Limits	Render Free sleeps after `15m` idle and uses ephemeral disk (`64` -session SQLite LRU).	Move pilot to always-on container + managed PostgreSQL + shared object audio cache.

10. One-Page Investor Quick-Reference Sheet

Dimension	Exact Fact / Metric (`v1.5.0`)	Status / Source
Core Architecture	FastAPI + SQLite (`5` pip deps, `~55.4 MB` RSS) + vanilla `kn` / `hi` / `en` JS	`[IMPLEMENTED]` (`12/12` preflight gates)
Does It Use RFM?	Computes exact **R** (`MIN(days_ago)`), **F** (`COUNT`), **M** (`SUM(amount)`) + **Cadence** (`median_gap`) & **Daypart**; uses rule thresholds today, not `1-5` quintiles	`[IMPLEMENTED]` (`engine.py:640` , `2029`); continuous `M` -hurdle RFM on roadmap
Synthetic Store Size	`240` buyers, `1,613` scans (`60d` GMV `Rs 3,06,365` ; `30d` GMV `Rs 1,74,685` / `917` scans)	`[SYNTHETIC]` (`seed = 20261025` , ending `25 Oct 2026`)
Repeat Revenue Share	`175 / 240` repeat buyers (`72.9%`) drive `Rs 2,89,175` (`94.4%` of 60d GMV)	`[SYNTHETIC]` (`loyal_champion` alone = `Rs 1,69,240`)
Churn & Rush Signals	`48` silent regulars (`40` eligible -> `32` offer / `8` holdout); `2` post-rush regulars (`Rs 4,920/mo = 2 x 205 x 12`); `-70%` `3-5 PM` drop (`30` vs `101` scans)	`[IMPLEMENTED]` SQL detectors (`OPS-CARE-02` inferred `"possibly after a busy rush"`)
Simulated 48h Return	`12/32` (`37.5%`) vs `1/8` (`12.5%`) -> `8` extra visits (`Rs 1,740` extra sales)	`[SIMULATED]` (`_draw_independent_outcomes` , `p = 0.2364`)
Simulated 48h Profit	**`+Rs 222.00`** pre-fee / **`+Rs 91.50`** post-5%-fee (`Rs 92` rounded in `Explain Simply`)	`[MODELLED Grade C]` (`30%` margin & `5%` fee assumptions)
Break-Even Margin	**`17.2%`** pre-fee / **`24.7%`** post-5%-fee (suits `~30%` darshinis first)	`[IMPLEMENTED]` `app/stats.py` (`6x4` sensitivity grid)
Runtime AI & Guards	**1. Sarvam** (`bulbul:v3` , `saaras:v4` , `sarvam-105b-conversations`) -> **2. Groq** (`gpt-oss-20b` , `whisper-large-v3-turbo` , `orpheus-v1-english`) -> **3. Browser Speech + Templates**	`[IMPLEMENTED]` + `assert_zero_pii` + 3-script `verify_numeric_grounding`
Variable COGS / Shop	**`Rs 48.00/mo` cached** (`Rs 71.80` uncached): `Rs 43` messaging + `Rs 3.20` TTS + `Rs 1.80` compute (`~Rs 5/mo` if native app push replaces WhatsApp)	`[MODELLED ASSUMPTION]` (`scripts/unit_economics.py` ; app does not send WhatsApp)
Pilot Infra Budget	`$0/mo` current Render Free demo; **`~Rs 4,000 - Rs 6,500/mo`** estimated all-in tech + API cost for a `50` -shop (`n = 2,000`) always-on pilot	`[INDICATIVE ESTIMATE]` (excludes `Rs 300/shop` promo cashback pool)
Test & Security Proof	`12` preflight gates, `7` test suites (`311` asserts), `21/21` QA battery, `0` leaked secrets, `DEMO_PIN` + `30/min` IP rate limit	`[IMPLEMENTED]` (`python3 scripts/preflight.py`)